Laravel and Middleware

Take a look back at Figure 10-1, at the start of this chapter.

We've covered the requests and responses, but we haven't actually looked into what middleware is. You may already be familiar with middleware; it's not unique to Laravel, but rather a widely used architecture pattern.

An Introduction to Middleware

The idea of middleware is that there is a series of layers wrapping around your application, like a multilayer cake or an onion. Just as shown in Example 10-1, every request passes through every middleware layer on its way into the application, and then the resulting response passes back through the middleware layers on its way out to the end user.

Middleware is most often considered separate from your application logic, and usually is constructed in a way that should theoretically be applicable to any application, not just the one you're working on at the moment.

Middleware can inspect a request and decorate it, or reject it, based on what it finds. That means middleware is great for something like rate limiting: it can inspect the IP address, check how many times it's accessed this resource in the last minute, and send back a 429 (Too Many Requests) status if a threshold is passed.

Because middleware also gets access to the response on its way out of the application, it's great for decorating responses. For example, Laravel uses a middleware to add all of the queued cookies from a given request/response cycle to the response right before it is sent to the end user.

But some of the most powerful uses of middleware come from the fact that it can be nearly the *first* and the *last* thing to interact with the request/response cycle. That makes it perfect for something like enabling sessions — PHP needs you to open the session very early and close it very late, and middleware is great for this.

Creating Custom Middleware

Let's imagine we want to have a middleware that rejects every request that uses the `DELETE` HTTP method, and also sends a cookie back for every request.

There's an Artisan command to create custom middleware. Let's try it out:

```
php artisan make:middleware BanDeleteMethod
```

You can now open up the file at *app/Http/Middleware/BanDeleteMethod.php*. The default contents are shown in Example 10-15.

Example 10-15. Default middleware contents

```
...
class BanDeleteMethod
{
    public function handle($request, Closure $next)
    {
        return $next($request);
    }
}
```

How this `handle()` method represents the processing of both the incoming request *and* the outgoing response is the most difficult thing to understand about middleware, so let's walk through it.

Understanding middleware's handle() method

First, remember that middleware are layered one on top of another, and then finally on top of the app. The first middleware that's registered gets *first* access to a request when it comes in, then that request is passed to every other middleware in turn, then to the app; then the resulting response is passed outward through the middleware, and finally this first middleware gets *last* access to the response when it goes out.

Let's imagine we've registered `BanDeleteMethod` as the first middleware to run. That means the `$request` coming into it is the raw request, unadulterated by any other middleware. Now what?

Passing that request to `$next()` means handing it off to the rest of the middleware. The `$next()` closure just takes that `$request` and passes it to the `handle()` method of the next middleware in the stack. It then gets passed on down the line until there are no more middleware to hand it to, and it finally ends up at the application.

Next, how does the response come out? This is where it might be hard to follow. The application returns a response, which is passed back up the chain of middleware — because each middleware returns its response. So, within that same `handle()` method, the middleware can decorate a `$request` and pass it to the `$next()` closure, and can then choose to do something with the output it receives before finally returning that output to the end user. Let's look at some pseudocode to make this clearer (Example 10-16).

Example 10-16. Pseudocode explaining the middleware call process

```
...
class BanDeleteMethod
{
    public function handle($request, Closure $next)
    {
        // At this point, $request is the raw request from the user.
        // Let's do something with it, just for fun.
        if ($request->ip() === '192.168.1.1') {
            return response('BANNED IP ADDRESS!', 403);
        }

        // Now we've decided to accept it. Let's pass it on to the next
        // middleware in the stack. We pass it to $next(), and what is
        // returned is the response after the $request has been passed
        // down the stack of middleware to the application and the
        // application's response has been passed back up the stack.
        $response = $next($request);

        // At this point, we can once again interact with the response
        // just before it is returned to the user
        $response->cookie('visited-our-site', true);

        // Finally, we can release this response to the end user
        return $response;
    }
}
```

Finally, let's make the middleware do what we actually promised (Example 10-17).

Example 10-17. Sample middleware banning the delete method

```
...
class BanDeleteMethod
{
    public function handle($request, Closure $next)
    {
        // Test for the DELETE method
        if ($request->method() === 'DELETE') {
            return response(
                "Get out of here with that delete method",
                405
            );
        }

        $response = $next($request);

        // Assign cookie
        $response->cookie('visited-our-site', true);

        // Return response
        return $response;
    }
}
```

Binding Middleware

We're not quite done yet. We need to register this middleware in one of two ways: globally or for specific routes.

Global middleware are applied to every route; route middleware are applied on a route-by-route basis.

Binding global middleware

Both bindings happen in *app/Http/Kernel.php*. To add a middleware as global, add its class name to the `$middleware` property, as in Example 10-18.

Example 10-18. Binding global middleware

```
// app/Http/Kernel.php
protected $middleware = [
    \Illuminate\Foundation\Http\Middleware\CheckForMaintenanceMode::class,
    \App\Http\Middleware\BanDeleteMethod::class,
];
```

Binding route middleware

Middleware intended for specific routes can be added as a route middleware or as part of a middleware group. Let's start with the former.

Route middleware are added to the `$routeMiddleware` array in *app/Http/Kernel.php*. It's similar to adding them to `$middleware`, except we have to give one a key that will be used when applying this middleware to a particular route, as seen in Example 10-19.

Example 10-19. Binding route middleware

```
// app/Http/Kernel.php
protected $routeMiddleware = [
    'auth' => \App\Http\Middleware\Authenticate::class,
    ...
    'nodelete' => \App\Http\Middleware\BanDeleteMethod::class,
];
```

We can now use this middleware in our route definitions, like in Example 10-20.

Example 10-20. Applying route middleware in route definitions

```
// Doesn't make much sense for our current example...
Route::get('contacts', [
    'middleware' => 'nodelete',
    'uses' => 'ContactsController@index'
]);

// Makes more sense for our current example...
Route::group(['prefix' => 'api', 'middleware' => 'nodelete', function () {
    // All routes related to an API
}]);
```

Using middleware groups

Laravel 5.2 introduced the concept of middleware groups. They're essentially pre-packaged bundles of middleware that make sense to be together in specific contexts.